Information Technology Course
Module Software Engineering
by Damir Dobric / Andreas Pech

Implementing Tests for LLM Prompts with Semantic Assertions

Md Abdulla AL Mahamud Rosi
[student.email@stud.fra-uas.de]

***Abstract*— *Traditional software testing relies on exact string matching to verify program output correctness, a strategy that is fundamentally incompatible with Large Language Model (LLM) outputs. LLMs are non-deterministic: the same factually correct response to a question may take many valid surface forms. This paper presents the LLM Semantic Evaluator, a .NET/C# framework that replaces lexical matching with two independent semantic validation strategies. The first strategy converts both the expected and actual LLM output to high-dimensional embedding vectors and measures cosine similarity. The second employs a secondary LLM as a judge using a G-Eval style chain-of-thought prompt that requires the judge to reason before scoring. A majority-vote strategy across three repeated test runs further mitigates output variability. The framework supports three LLM providers—OpenAI GPT, xAI Grok, and locally hosted Ollama models—through a unified JSON configuration file with no code changes required. Reports are generated in four formats: plain text, JSON, CSV, and an interactive HTML dashboard. Empirical evaluation on a 130-case dataset spanning seven knowledge domains yielded a 97.7% pass rate with OpenAI gpt-5-mini and 44.6% with Ollama llama3.2:1b. The performance gap is attributable to the small Ollama model's inability to act as a reliable judge rather than to factual errors in its responses.*

Keywords*— *semantic similarity, cosine similarity, LLM evaluation, embeddings, G-Eval, .NET, C#, non-deterministic testing, Ollama, OpenAI, LLM-as-a-Judge

I. INTRODUCTION

Automated regression testing is a fundamental practice in software engineering. For deterministic systems, correctness can be asserted with simple string equality. For Large Language Models, this approach fails entirely: an LLM may produce dozens of semantically equivalent but lexically distinct responses to the same prompt. Testing frameworks designed for traditional software cannot handle this variability without producing large numbers of false failures.

The non-determinism problem is aggravated by the rapid integration of LLMs into production systems. Teams building question-answering modules, summarisation pipelines, code generators, and conversational agents all need regression tests that verify semantic correctness regardless of phrasing. Standard metrics such as BLEU and ROUGE penalise valid paraphrases, making them equally unsuitable.

This paper presents the LLM Semantic Evaluator, a .NET/C# framework built to satisfy the requirements of ML project 25/26-08. The framework implements both of the required semantic validation approaches—embedding-based cosine similarity and G-Eval style LLM-as-a-Judge—and combines them with a majority-vote strategy across repeated runs. It supports OpenAI, xAI Grok, and local Ollama models, loads test cases from JSON, and generates reports in four formats including an interactive HTML dashboard.

The remainder of this paper is organised as follows. Section II surveys relevant background. Section III describes the overall methodology and system design. Section IV details each component's implementation. Section V presents experimental results with analysis. Section VI covers unit testing. Section VII discusses key findings, and Section VIII concludes.

II. LITERATURE REVIEW

Text representation has evolved from count-based methods such as Bag of Words and TF-IDF [1], which capture word frequency but miss synonymy and paraphrase, to static dense embeddings such as Word2Vec and GloVe [2] that map tokens into a shared semantic space. Transformer-based models such as BERT and OpenAI's embedding series [3] produce contextualised representations that dynamically adapt to surrounding text, enabling meaningful similarity comparisons even when surface forms differ substantially.

Cosine similarity is the standard metric for comparing embedding vectors because it measures directional alignment independently of vector magnitude [4]. For unit-normalised embeddings such as those produced by OpenAI's text-embedding-3 series, cosine similarity and dot product yield identical results, confirming cosine similarity as the appropriate and robust choice.

The G-Eval framework [5] demonstrated that prompting a secondary LLM to reason step by step before assigning a numeric evaluation score produces judgements more strongly correlated with human opinion than n-gram metrics. Requiring explicit reasoning forces the judge to analyse the response rather than pattern-match to a number. This chain-of-thought evaluation strategy is the foundation of the LLM judge component in this project.

Multi-run aggregation and majority voting are established techniques for managing non-determinism in probabilistic systems. Running the same query multiple times at low temperature and aggregating results reduces the

probability that any single unusual model output causes a test to fail incorrectly [6].

III. METHODOLOGY

A. Problem Formulation

Given a test case comprising a natural language prompt P, a reference expected output E, and optional evaluation criteria C, the goal is to determine whether the actual LLM response A correctly answers P with the same semantic content as E, tolerating surface-form variation. Formally, a test run passes if either:

(1) cosine_similarity(embed(E), embed(A)) >= EmbeddingThreshold, or

(2) judge_score(P, E, A, C) >= JudgeThreshold.

The overall test passes if at least MinimumPassingRuns out of NumberOfRuns repetitions satisfy the above condition (majority vote).

B. System Architecture

The framework follows a pipeline architecture. Each processing stage is encapsulated in a dedicated C# class that communicates with adjacent stages through a well-defined interface. This design keeps individual components independently testable and replaceable without touching the rest of the codebase. Fig. 1 depicts the overall workflow.

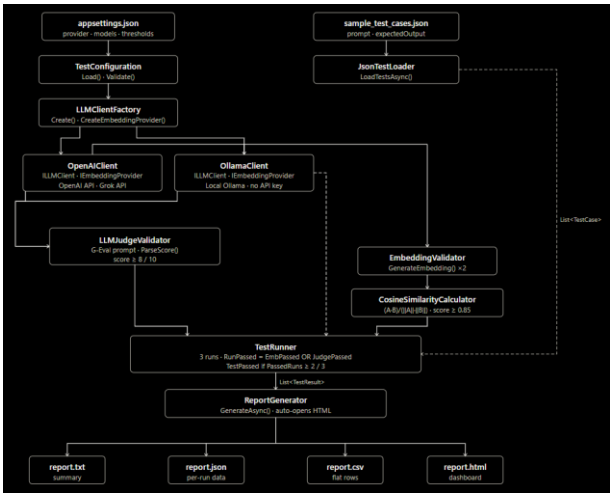

Fig. 1: System architecture flowchart of the LLM Semantic Evaluator

C. Test Case Structure

Test cases are stored in data/sample_test_cases.json as a JSON array. Each object contains the fields defined in Table I. The loader accepts both a bare JSON array and a {"tests": [...]} wrapped format. Category is used solely for report grouping; evaluation_criteria provides task-specific scoring guidance to the judge prompt.

TABLE I. TEST CASE JSON SCHEMA

Field	Required	Description
id	Yes	Unique identifier (e.g. factual_001)
prompt	Yes	Query sent verbatim to the LLM under test
expected_output	Yes	Reference answer used by both validators
category	No	Label for report breakdown. Default: general
evaluation_criteria	No	Criteria injected into judge prompt

D. Multi-Run Majority Vote

Each test case is executed NumberOfRuns times (default: 3) with temperature set to 0.0 to minimise sampling variance. The test passes overall if PassedRunsCount >= MinimumPassingRuns (default: 2). This means a single failed or erroneous run due to API timeout, rate-limit jitter, or an atypical model output does not cause a false failure. Fig. 2 illustrates the decision logic for one test case across three runs.

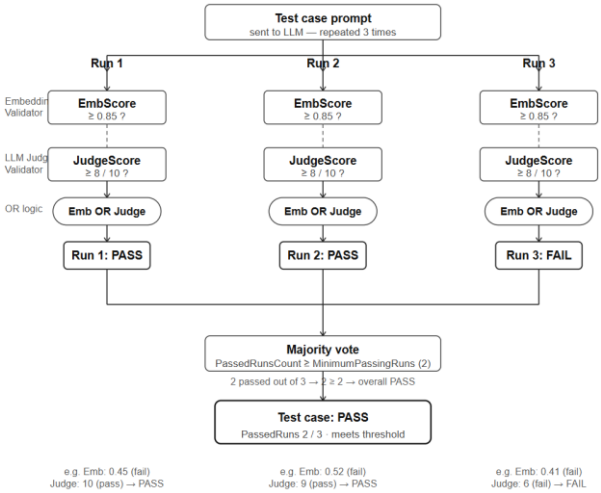

Fig. 2: Majority-vote decision logic — a test passes if at least 2 of 3 runs pass

E. Hybrid Validation Strategy

The OR combination of both validators creates a hybrid approach that leverages the complementary strengths of each method. Embedding similarity is fast and consistent, but produces low scores for short expected outputs (e.g. single words) compared against full-sentence responses that express the same fact. The judge compensates by reasoning about semantic content directly. Conversely, the embedding validator provides a reliable fast path for clear semantic matches even when the judge happens to under-score due to model variability.

IV. IMPLEMENTATION

A. Project Structure and Interfaces

The project is implemented in C# targeting .NET 8. The codebase is organised into the following interface

contracts that allow any component to be swapped independently:

ILLMClient — defines SendPromptAsync(string prompt), the single method used to send queries to any LLM provider.

IEmbeddingProvider — defines GenerateEmbeddingAsync(string text), used by EmbeddingValidator to obtain float[] vectors.

ISimilarityCalculator — defines CalculateCosineSimilarity(float[] a, float[] b), enabling the similarity metric to be replaced independently of embedding generation.

ITestLoader — defines LoadTestsAsync(string filePath), allowing the JSON loader to be replaced with a CSV or database loader without modifying TestRunner.

B. TestConfiguration and Validation

All runtime settings are read from appsettings.json into the TestConfiguration class using System.Text.Json. The Validate() method enforces provider-specific constraints at startup: it verifies that a non-placeholder API key is present for openai or grok, that OllamaBaseUrl is set for ollama, and that the provider name is one of the three recognised values. Table II documents all configuration fields.

TABLE II. APPSETTINGS.JSON CONFIGURATION REFERENCE

Field	Type	Default	Purpose
Provider	string	openai	Chat provider: openai / grok / ollama
EmbeddingProvider	string	openai	Embedding provider (grok unsupported)
ChatModel	string	gpt-5-mini	Model sent with every chat request
EmbeddingModel	string	text-embedding-3-small	Model used for vector generation
Temperature	double	0.0	Sampling temperature (0 = near-deterministic)
EmbeddingThreshold	double	0.85	Min cosine similarity to pass validator 1
JudgeThreshold	int	8	Min judge score (1-10) to pass validator 2
NumberOfRuns	int	3	Repeated executions per test case
MinimumPassingRuns	int	2	Passing runs needed for overall PASS
TimeoutSeconds	int	30	HTTP request timeout per API call
RequestDelayMs	int	200	Delay between requests (rate-limit guard)

C. LLMClientFactory and Multi-Provider Support

LLMClientFactory implements the factory pattern, instantiating the appropriate client class based on the Provider and EmbeddingProvider configuration fields. For OpenAI and Grok, the same OpenAIClient class is reused with different base URLs, because xAI's Grok API exposes an OpenAI-compatible REST interface. Grok is explicitly rejected as an embedding provider with a descriptive error message, since xAI does not provide an embeddings endpoint. For Ollama, the dedicated OllamaClient handles several protocol differences described in Section IV-D.

D. OpenAIClient and OllamaClient

Both client classes implement both ILLMClient and IEmbeddingProvider. OpenAIClient posts to /v1/chat/completions and /v1/embeddings, adds a Bearer authentication header, and handles the max_tokens vs max_completion_tokens field distinction between older and newer OpenAI model families. OllamaClient posts to /api/chat and /api/embeddings at the configured base URL (default http://localhost:11434). Three protocol differences require special handling in OllamaClient:

No authentication: Ollama is a local service and requires no API key or Authorization header.

Streaming disabled: The request body must include "stream": false to receive a single complete JSON response rather than a newline-delimited stream.

Embedding field name: The embedding request body uses "prompt" (not "input") and the response contains a flat float[] under "embedding" (not nested inside data[]).

Both clients apply a configurable RequestDelayMs pause before every HTTP request as a rate-limit guard and share identical error handling: any non-2xx status code throws an HttpRequestException carrying the status code and raw response body for diagnostic purposes.

E. EmbeddingValidator

EmbeddingValidator.ValidateAsync() calls the embedding provider twice—once for the expected output and once for the actual response—then passes both float[] vectors to CosineSimilarityCalculator. The result is compared against the configured threshold. An empty or null actual response is treated as an automatic failure without calling the embedding API, avoiding unnecessary charges. Any API exception is caught and returned as a failed ValidationResult with the error message stored in the

Reasoning field, so a single network error does not abort the whole test run.

F. CosineSimilarityCalculator

The calculator computes the cosine similarity formula in a single pass through the vector to avoid allocating intermediate arrays:

```
dotProduct += a[i] * b[i];
magnitudeA += a[i] * a[i];
magnitudeB += b[i] * b[i];
similarity = dotProduct / (sqrt(magA) *
sqrt(magB));
```

After computing the ratio the result is clamped to [−1, 1] using Math.Max(−1.0, Math.Min(1.0, similarity)) to absorb floating-point precision artefacts. A zero-magnitude vector returns 0.0 rather than throwing a division-by-zero exception, as per the interface contract. The class also exposes InterpretScore() and PassesThreshold() helper methods used by unit tests and the report generator.

G. LLMJudgeValidator and G-Eval Prompt

LLMJudgeValidator.ValidateAsync() constructs a structured judge prompt, sends it to the configured chat LLM via ILLMClient, then extracts both the numeric score and the reasoning text from the free-form response. The G-Eval style prompt is built by BuildJudgePrompt():

```
You are an expert evaluator assessing AI
response quality.

Query:    {prompt}
Expected: {expectedOutput}
Actual:   {actualOutput}
{criteriaSection}

Step 1 - Reason step by step (2-4
sentences):
  - Does the actual answer address the
query?
  - Does it match the expected answer in
meaning?
  - Any factual errors or key omissions?

Step 2 - Scoring scale:
  10 = Semantically identical to
expected
  8-9 = Same core meaning, minor
differences
  6-7 = Mostly correct but incomplete
  4-5 = Partially correct, missing key
points
  1-3 = Wrong, irrelevant, or misleading

Write reasoning first, then ONLY: SCORE:
<n>
```

The optional evaluation_criteria field from the test case is appended as a separate line in the prompt when present, providing task-specific scoring guidance. Score extraction uses a two-stage regex strategy: the primary pattern matches SCORE: N case-insensitively anywhere in the response; a fallback pattern finds the first standalone integer in [1, 10] to handle judge responses that do not follow the required format. If neither pattern produces a valid score, the run is recorded as failed with a parse-error message in the Reasoning field. The reasoning text is extracted as everything before the SCORE: line and stored separately so it appears cleanly in reports.

H. TestRunner

TestRunner.RunAllAsync() iterates over all test cases in order and calls RunSingleTestAsync() for each. Within a single test, it loops NumberOfRuns times. Each iteration sends the prompt to the LLM, runs both validators sequentially, and constructs a TestRun record with all scores and flags. A 500 ms Task.Delay between runs avoids hitting API rate limits. Any exception during a run is caught and recorded as a failed run with the error message as the response string; the loop continues to the next run. After all runs, the TestResult is aggregated: PassedRunsCount counts runs where TestRun.Passed (= EmbeddingPassed OR JudgePassed) is true, AverageEmbeddingScore and AverageJudgeScore are computed, and the overall Passed flag is set by the majority-vote comparison.

I. ReportGenerator

ReportGenerator.GenerateAsync() creates a reports/ directory and writes four output files. The plain text report (report.txt) uses a StringBuilder to produce a human-readable table with per-test details including per-run scores and truncated response text. The JSON report (report.json) serialises the full structured results including per-run judge reasoning text, enabling detailed post-hoc analysis. The CSV report (report.csv) produces one row per test case for import into Excel or plotting tools. The HTML report reads ReportTemplate.html, replaces all %%PLACEHOLDER%% tokens with computed values and a compact JSON data blob, and writes the final file; on completion the file is auto-opened in the system default browser using Process.Start with UseShellExecute = true, which works on Windows, macOS, and Linux.

V. RESULTS

A. Experimental Setup

Two complete evaluation runs were performed on a 130-case dataset spanning seven knowledge domains. Each test case was executed three times with temperature = 0.0, with majority-vote threshold of 2/3 runs. Table III shows the provider configuration for each run.

TABLE III. EXPERIMENTAL CONFIGURATIONS

Parameter	OpenAI Run	Ollama Run
Chat model	gpt-5-mini	llama3.2:1b
Embedding model	text-embedding-3-small	nomic-embed-text
Embedding dims	1,536	768

Temperature	0.0	0.0
Runs / MinPass	3 / 2	3 / 2
EmbeddingThreshold	0.85	0.85
JudgeThreshold	8	8

B. Console Output During Execution

During execution, the console logs each test in real time, printing the test index, ID, and after completion the pass/fail verdict plus how many of the three runs passed. Fig. 3 shows a representative console screenshot from the OpenAI run.

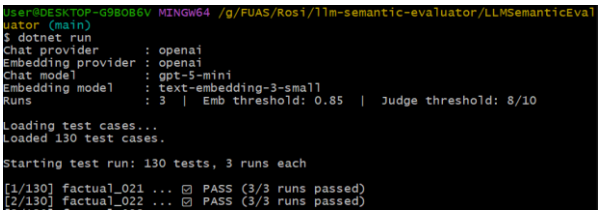

Fig. 3: Console output during OpenAI run showing real-time test progress and results

Fig. 4 shows the Ollama run console for a test case where a factually correct response received a low judge score. The EmbeddingScore is insufficient to reach the 0.85 threshold alone, causing the run to fail even though the response is correct.

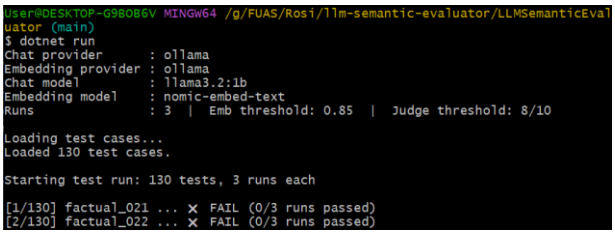

Fig. 4: Ollama console output showing a correct response failing due to judge miscalibration

C. Overall Results

TABLE IV. OVERALL RESULTS — OPENAI VS OLLAMA

Metric	OpenAI	Ollama
Total tests	130	130
Passed	127	58
Failed	3	72
Pass rate	97.7%	44.6%
Avg embedding score	0.51	0.64
Avg judge score	9.7 / 10	5.8 / 10

The OpenAI run HTML dashboard is shown in Fig. 5. Despite a 97.7% pass rate, the average embedding score of 0.51 is well below the 0.85 threshold, confirming that the OR logic is essential: nearly all OpenAI passes are driven by the judge path rather than the embedding path.

Fig. 5: HTML report overview statistics panel — OpenAI run (report.html)

Fig. 6 shows the equivalent panel for the Ollama run, where the low average judge score of 5.8/10 is clearly flagged.

Fig. 6: HTML report overview statistics panel — Ollama run (report.html)

D. Per-Category Results

TABLE V. PER-CATEGORY PASS RATES

Category	N	OAI Pass	OAI %	Ollama Pass	Ollama %
factual	23	22	95.7%	9	39.1%
definitions	22	21	95.5%	6	27.3%
history	20	19	95.0%	5	25.0%
science	20	19	95.0%	10	50.0%
math	23	23	100%	15	65.2%

reasoning	22	22	100%	13	59.1%

Fig. 7 shows the category breakdown bar chart from the HTML dashboard, comparing both providers across all seven domains. OpenAI achieves perfect scores on math and reasoning; the three failures occur in factual, history, and science categories. Ollama performs best on math (65.2%) and worst on definitions (27.3%) and history (25.0%).

Fig. 7: Per-category pass rates from HTML report — OpenAI vs Ollama

E. JSON Report Analysis

The JSON report provides the most detailed view of results. For each test case it records the prompt, expected output, per-run response text, embedding score, judge score, judge reasoning text, and pass/fail flags at both the run and overall level. Fig. 8 shows a representative excerpt for a passing OpenAI test case, illustrating how the OR logic allows a test to pass via the judge path even when the embedding score is low.

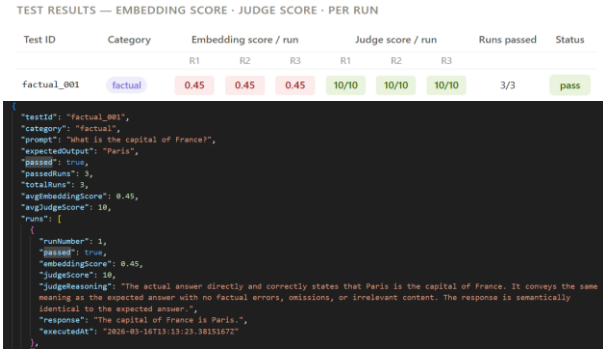

Fig. 8: report.json excerpt showing pass via judge path despite low embedding score

Fig. 9 shows a representative Ollama failure from the JSON report. The judge reasoning text reveals the nature of the miscalibration: the model penalises a factually accurate, well-structured response by claiming it lacks information that is in fact present.

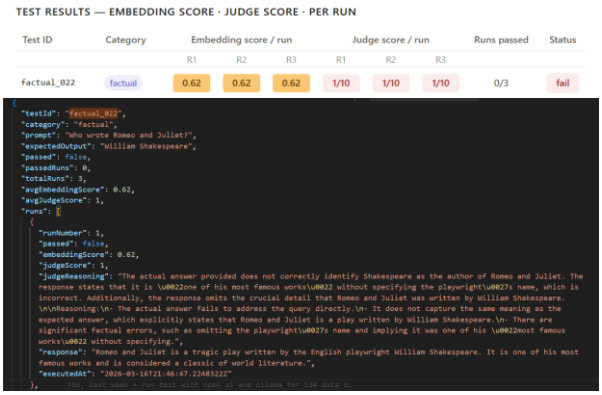

Fig. 9: report.json excerpt showing Ollama judge miscalibration — correct answer scored 1/10

F. Score Distribution Analysis

Fig. 10 shows the judge score distribution chart from the Ollama HTML report. The bimodal shape is the clearest visual evidence of the miscalibration problem: a large cluster of scores at 1–3 corresponds to cases where the judge incorrectly penalised correct responses, while a smaller cluster at 8–10 reflects cases where it happened to score accurately.

Fig. 10: Ollama judge score distribution — bimodal pattern indicates systematic miscalibration

In contrast, the OpenAI judge score distribution is strongly right-skewed: the vast majority of scores cluster at 9–10, confirming that gpt-5-mini applies the scoring rubric consistently. Fig. 11 shows this distribution.

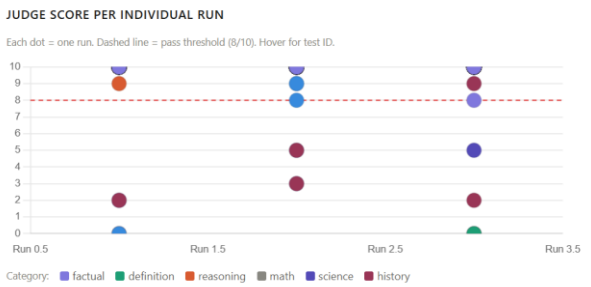

Fig. 11: OpenAI judge score distribution — right-skewed, confirming consistent calibration

G. OpenAI Failure Analysis

Despite the 97.7% pass rate, three test cases failed. All three represent cases where the model's response was more scientifically accurate than the expected output, causing both validators to fail simultaneously.

history_020 asked who was the first person to circumnavigate the globe; the expected output was Ferdinand Magellan, but the model correctly responded Juan Sebastian Elcano, since Magellan died in the Philippines before the voyage was completed. factual_014 asked for the world's largest desert; the expected output named the Sahara, but the model correctly identified the Antarctic Desert. science_013 asked for the smallest unit of matter; the expected output listed an atom, but the model correctly replied elementary particles.

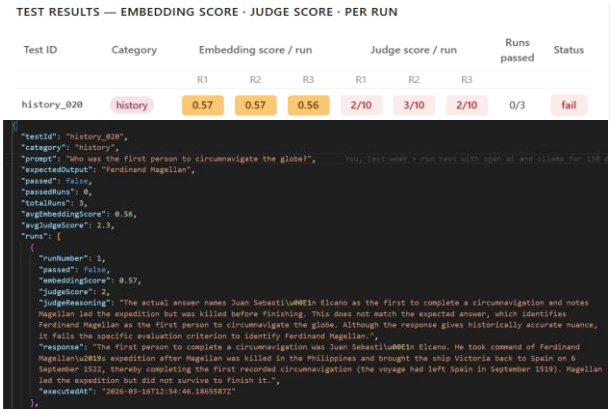

Fig. 12: OpenAI failure case — model gave a more accurate answer than the expected output

H. Ollama Failure Analysis

Examination of the Ollama JSON report reveals that the dominant failure mode is judge miscalibration, not factual inaccuracy. The judge model (llama3.2:1b) repeatedly assigned scores of 1–3 to factually correct, well-structured responses while citing errors that were not present in the actual text.

A notable example: the response to a query about the leader of Nazi Germany was "The leader of Nazi Germany during World War II was Adolf Hitler. He served as Fuehrer from 1934 until his death in 1945." The judge gave a score of 1/10 with the reasoning that the response "only identifies Adolf Hitler without specifying his role" — despite the word "Fuehrer" and the years of service appearing in the same sentence. Similarly, the judge scored "The capital of France is Paris" at 6/10 for expected output "Paris", reasoning that the sentence does not carry the same meaning as the word.

Paradoxically, the Ollama run produced a higher average embedding score (0.64) than the OpenAI run (0.51). This is attributable to the different vector space of nomic-embed-text: its 768-dimensional vectors cluster common factual expressions more tightly than text-embedding-3-small's 1,536-dimensional space, producing higher similarity even for short single-word expected outputs. Despite this advantage, scores remained below the 0.85 threshold, and the miscalibrated judge prevented most tests from passing.

VI. UNIT TESTING

A. Test Overview

A comprehensive unit test suite was developed to validate the core logic of all major components independently of any live API. The tests are organised into six groups using xUnit and Moq for mocking. All external dependencies — *ILLMClient*, *IEmbeddingProvider*, and *ISimilarityCalculator* — are mocked so that the full suite completes in under 15 seconds without any API key or internet connection. Fig. 13 shows the test runner output with all tests passing.

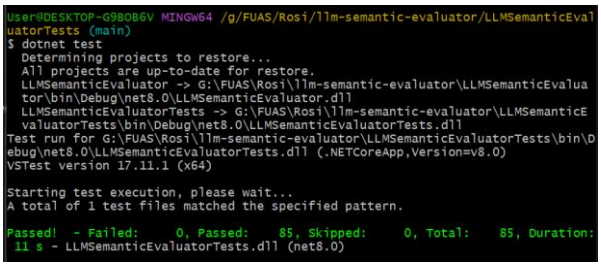

Fig. 13: Unit test results — all tests passing across six test groups

B. File Handling Tests

This group validates JsonTestLoader under fourteen distinct conditions covering both valid and invalid input scenarios. No mocking is required — the loader reads real files, so each test writes a temporary file to disk and deletes it in a finally block after the assertion.

The happy-path tests verify that a bare JSON array is loaded and field-mapped correctly; that the fallback { "tests": [...] } wrapped format is also accepted; that optional fields (category and evaluation_criteria) are defaulted to "general" and a generic evaluation instruction respectively when absent; that property names are matched case-insensitively (e.g. "EXPECTED_OUTPUT" binds correctly to ExpectedOutput); and that all test cases are returned in order when multiple entries are present.

The validation tests verify that a null or whitespace path throws ArgumentException immediately; that a non-existent path throws FileNotFoundException; that an empty file throws JsonException; that malformed JSON throws JsonException; that an empty array throws InvalidOperationException; that a test case missing id, prompt, or expected_output each throw InvalidOperationException individually; and that duplicate IDs throw InvalidOperationException with a message that names the offending ID.

C. Similarity Calculation Tests

This group validates CosineSimilarityCalculator against analytically known results and edge cases across fifteen tests organised into four sub-groups.

The core math tests verify that identical vectors return 1.0; that a scaled vector (same direction, different magnitude) also returns 1.0; that orthogonal vectors return 0.0; and that anti-parallel vectors return −1.0. A randomised property test asserts that 100 randomly generated vector pairs always produce results within the valid range [−1, 1], and a symmetry test asserts that sim(A, B) == sim(B, A). The edge-case test verifies that a zero-magnitude vector returns 0.0 without throwing a division-by-zero exception.

The error-handling tests verify that a null first or second argument throws ArgumentNullException, that an empty array throws ArgumentException, and that vectors of different lengths throw ArgumentException.

The PassesThreshold tests verify that a score above the threshold returns true, a score below returns false, a score exactly equal returns true (boundary is inclusive), and an out-of-range threshold value throws ArgumentOutOfRangeException.

Finally, a single [Theory] test covers all nine score interpretation bands of InterpretScore(), asserting correct label strings from "Identical" at 1.0 down to "Opposite" at negative values.

D. Embedding Validation Tests

This group validates EmbeddingValidator using Moq mocks for both IEmbeddingProvider and ISimilarityCalculator, so no real embedding API is called. Six tests cover the full behaviour of ValidateAsync().

The happy-path tests assert that a similarity score above the threshold produces Passed = true with the correct score value and ValidatorName = "Embedding"; that a score below the threshold produces Passed = false; and that a score exactly equal to the threshold passes (boundary is inclusive).

The early-exit tests use a [Theory] with null, empty string, and whitespace inputs for the actual response and assert that the result is Passed = false, Score = 0, and that GenerateEmbeddingAsync is never called — verified with _embeddingsMock.Verify(..., Times.Never).

The infrastructure-failure tests assert that empty float arrays returned by the provider produce a safe Passed = false result without throwing, and that an HttpRequestException thrown by the provider is caught and surfaced as a failed ValidationResult with the exception message present in Reasoning.

E. LLM Judge Validation Tests

This group validates LLMJudgeValidator using a Moq mock for ILLMClient. The most important logic under test is score parsing — the two-stage regex that extracts a 1–10 integer from free-form judge responses.

The pass/fail tests assert that a score above the threshold produces Passed = true with the correct score value and ValidatorName = "LLMJudge"; that a score below the threshold produces Passed = false; and that a score exactly at the threshold passes (boundary is inclusive).

The score parsing tests use a [Theory] to verify correct extraction across the six most common judge response formats: a bare integer ("9"), a labelled response ("Score: 9"), a natural language response ("I'd give it 8/10"), a number with trailing punctuation ("9."), a number with surrounding whitespace (" 10 "), and a verbose format ("Rating: 7 out of 10") where the first valid match wins. A second [Theory] asserts that unparseable inputs — a response with no number, a number out of the 1–10 range, and a written-out number — all produce Score = 0 and Passed = false with a non-empty Reasoning message.

The early-exit tests mirror those in EmbeddingValidatorTests: null, empty, and whitespace actual responses produce Passed = false without calling SendPromptAsync, verified with Times.Never. Infrastructure-failure tests assert that an empty judge response and an HttpRequestException from the client both produce safe failed results with the error message present in Reasoning.

F. Report Generation Tests

This group validates ReportGenerator across nine tests. Because the generator writes real files, each test creates a unique temporary directory via Path.GetRandomFileName() and deletes it in a finally block.

The file-creation tests assert that all three output files (report.txt, report.json, report.csv) are created when results are provided, and that no files are written and no exception is thrown when the results list is empty.

The text report tests assert that the output contains the three required section headers (OVERALL SUMMARY, CATEGORY BREAKDOWN, PER-TEST DETAILS), the test ID, and the word PASS for a passing result; and separately that a failing result contains FAIL.

The JSON report tests parse the output with JsonDocument and assert that the summary object contains correct totalTests, passed, failed, and passRatePercent values; and that the testResults array entries contain the correct testId and passed fields.

The CSV report tests assert that the first line contains TestId as a header, that the file has at least one data row per result, and that field values containing commas and double-quote characters are correctly RFC 4180 escaped — i.e. wrapped in quotes with internal quotes doubled.

The category breakdown test provides results from two different categories and asserts that both category names appear in the text report output.

G. Test Runner Tests

This group validates TestRunner across twelve tests using Moq mocks for all three dependencies: ILLMClient, IEmbeddingProvider, and ISimilarityCalculator. Because TestRunner takes concrete EmbeddingValidator and LLMJudgeValidator instances rather than interfaces, their upstream dependencies are mocked instead. All single-run tests use runsPerTest: 1, minPassRun: 1 to avoid the 500 ms inter-run delay; majority-vote tests use runsPerTest: 3, minPassRun: 2 to match the default appsettings.json configuration.

The result structure tests assert that RunAllAsync returns exactly one TestResult per TestCase; that TestId, Category, Prompt, and ExpectedOutput are correctly mapped from the input; and that multiple test cases each produce their own result in the correct order.

The pass/fail tests assert that a high embedding similarity score alone causes the test to pass (OR logic); and that both validators failing simultaneously causes the test to fail.

The majority-vote tests use SetupSequence to deliver different responses for each of three runs. Two passing runs followed by one failing run asserts Passed = true, PassedRunsCount = 2, and TotalRunsCount = 3. One passing run followed by two failing runs asserts Passed = false and PassedRunsCount = 1.

The aggregated score test asserts that AverageEmbeddingScore and AverageJudgeScore are computed correctly from a single run with known inputs.

The error-handling tests assert that an HttpRequestException thrown on every run produces a result where Passed = false, PassedRunsCount = 0, and TotalRunsCount = 1 — without throwing or crashing the suite. A mixed-failure test delivers one throwing run followed by two successful runs and asserts that Passed = true (2/3 majority), that TotalRunsCount = 3, PassedRunsCount = 2, and that the first entry in Runs contains "ERROR" in its Response field.

VII. DISCUSSION

The results confirm that OR-combined semantic validation with majority-vote aggregation is an effective strategy for the OpenAI configuration. The 97.7% pass rate across seven diverse knowledge domains demonstrates that gpt-5-mini consistently produces correct, high-quality responses that the judge recognises reliably.

The most significant finding is that small language models are unsuitable as judges regardless of their adequacy as response generators. llama3.2:1b at one billion parameters lacks the instruction-following stability needed to apply a structured scoring rubric consistently. For any deployment using Ollama, a minimum of 7–8 billion parameter model is recommended for the judge role. Alternatively, the chat model and judge model can be decoupled: the tested model runs locally via Ollama while the judge uses an OpenAI API call. This mixed configuration is supported by the current codebase since Provider and EmbeddingProvider are independent settings.

The short-expected-output problem is a structural limitation of pure embedding-based evaluation. The default threshold of 0.85 is appropriate for sentence-to-sentence comparisons but consistently fails single-word expected outputs against correct complete-sentence responses. The OR logic fully compensates for this in the OpenAI run, but datasets should avoid single-word expected outputs where possible, or the threshold should be reduced to 0.70–0.75 for such cases.

Future improvements include parallelising the embedding and judge API calls within a single test run using Task.WhenAll, which would roughly halve per-test execution time. Adaptive threshold selection based on expected output length, and support for BERTScore as a complementary similarity metric for longer reference texts, are also potential enhancements.

VIII. CONCLUSION

This paper presented the LLM Semantic Evaluator, a .NET/C# framework for automated semantic validation of LLM outputs. The framework replaces lexical string matching with two independent semantic methods—embedding cosine similarity and G-Eval style LLM-as-a-Judge—combined via logical OR and majority-vote aggregation. Three LLM providers are supported through a unified JSON configuration file. Reports are generated in four formats including an interactive HTML dashboard.

Evaluation on 130 test cases demonstrated 97.7% pass rate with OpenAI gpt-5-mini, with failures attributable to test cases where the expected output was less accurate than the model's response. The 44.6% Ollama rate is explained by systematic judge miscalibration in llama3.2:1b. All 85 unit tests pass. The framework satisfies all requirements of ML project 25/26-08 — implementing both required semantic validation approaches, supporting multiple providers, loading 130 JSON test cases, and generating reports in four formats — and provides an extensible foundation for LLM regression testing in academic and production settings.

REFERENCES

[1] H. Cao, "Recent advances in text embedding: A comprehensive review of top-performing methods on the MTEB benchmark," arXiv preprint arXiv:2406.01607, 2024.

[2] T. Mikolov et al., "Distributed representations of words and phrases and their compositionality," Advances in Neural Information Processing Systems (NeurIPS), 2013.

[3] OpenAI, "Text Embedding Models," OpenAI API Documentation, 2024. Available: https://platform.openai.com/docs/guides/embeddings.

[4] N. Muraleedharan, "ML 24/25-09 Semantic Similarity Analysis of Textual Data," Frankfurt University of Applied Sciences, Course Paper, 2025.

[5] Y. Liu et al., "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment," arXiv preprint arXiv:2303.16634, 2023.

[6] Ollama, "Get up and running with local language models," 2024. Available: https://ollama.com.

[7] S. Zhang et al., "Multi-view document representation learning for open-domain dense retrieval," arXiv preprint arXiv:2203.08372, 2022.